

Laboratório de Concorrência • Aula 10

Maria Eduarda de Castro Lachimia • RA 24055202-2 • Turma A

Gerado em 09/10/2026, 22:10:54 • tempo registrado no cronômetro: 68:18

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

sincronização de cooperação: acontece quando duas ou mais pessoas precisam trabalhar juntas para realizar uma tarefa, exemplo: duas pessoas montando um móvel, uma segura as peças e a outra monta

competição: acontece quando duas ou mais pessoas querem utilizar o mesmo recurso mas apenas uma pode utilizar por vez, exemplo: várias pessoas querem utilizar o banheiro e cada uma tem que esperar a sua vez

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

acredito que vai imprimir 2.000.000 já que cada uma soma 1.000.000 cada né

1ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1215438

2ª execução (n = 1.000.000)

// Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549) // Cole em <https://onecompiler.com/java> public class Main { static int contador = 0; // variável COMPARTILHADA public static void main(String[] args) throws InterruptedException { int n = 1_000_000; // repetições de cada thread Runnable incrementa = () -> { for (int i = 0; i < n; i++) contador++; }; 986 ms esperado: 2000000 obtido: 1020600

3ª execução (n = 1.000.000)

esperado: 2000000 obtido: 1967358

Saída do programa

não respondido



```
1 // Prática 1 - Condição de corrida (Sebesta 13.2.1, p. 547-549)
2 // Cole em https://onecompiler.com/java
3 public class Main {
4     static int contador = 0; // variável COMPARTILHADA
5
6     public static void main(String[] args) throws InterruptedException {
7         int n = 1_000_000; // repetições de cada thread
8         Runnable incrementa = () -> {
9             for (int i = 0; i < n; i++) contador++;
10        };
11        Thread t1 = new Thread(incrementa);
12        Thread t2 = new Thread(incrementa);
13        t1.start(); t2.start(); // as duas executam ao mesmo tempo
14        t1.join(); t2.join(); // espera as duas terminarem
15        System.out.println("esperado: " + 2 * n);
16        System.out.println("obtido: " + contador);
17    }
18 }
```

Console

esperado: 2000000
obtido: 1967358

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

não, os valores ficaram abaixo dos 2.000.000 esperados e foram diferentes

isso acontece porque as duas threads acessam o mesmo contador ao mesmo tempo, fazendo com que algumas somas se percam

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

acontece quando as duas threads leem o mesmo valor, por exemplo, 5. a thread A soma 1 e a B também, ficando com 6. quando as duas gravam o resultado o contador fica em 6 sendo que deveria ser 7, assim uma das atualizações acaba sendo perdida

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

com $n = 1000$ o resultado foi 2.000 nas duas execuções, ou seja deu certo

isso é perigoso porque durante os testes o programa pode funcionar normalmente mas quando for utilizado de verdade com mais operações acontecendo ao mesmo tempo, podem aparecer erros que não foram percebidos antes

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

não respondido

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

são 20 ordens possíveis e 4 resultados diferentes: 4, 6, 7 e 8

dessas 20 ordens, apenas 1 deu o resultado correto que é 8

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

na ordem AABBBB, a tarefa A lê o valor 3 e soma 1, ficando com 4

depois a tarefa B lê o valor 3, multiplica por 2 e grava 6

por último, a tarefa A grava o valor 4 que tinha calculado antes substituindo o 6 assim a operação de B acaba sendo perdida

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

acredito que a versão sem sincronização vai ser a mais rápida porque não precisa controlar o acesso das threads mas também acho que ela pode apresentar um resultado incorreto enquanto as outras duas devem funcionar corretamente

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1032634 em 4 ms
synchronized:      2000000 em 84 ms
AtomicInteger:      2000000 em 23 ms
```

```
esperado:          2000000
sem sincronizacao: 1077580 em 4 ms
synchronized:      2000000 em 87 ms
AtomicInteger:      2000000 em 71 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

as versões synchronized e AtomicInteger deram o resultado correto de 2.000.000 nas duas execuções. A versão sem sincronização foi a mais rápida, levando 4 ms nas duas vezes, mas apresentou resultados errados. Já a synchronized levou 84 e 87 ms, enquanto a AtomicInteger levou 23 e 71 ms. isso acontece porque as versões corretas precisam controlar o acesso das threads ao contador, o que acaba levando mais tempo

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

não, porque o AtomicInteger controla cada saldo separadamente, mas não garante que as duas operações aconteçam juntas. assim, poderia acontecer de o dinheiro sair de uma conta e não chegar na outra. para evitar isso, seria necessário garantir que a transferência inteira seja realizada, ou que nenhuma das operações aconteça

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

acredito que não vão aparecer porque a thread A vai ficar esperando o recurso Y, enquanto a thread B vai ficar esperando o recurso X. como nenhuma das duas consegue liberar o recurso que a outra precisa, o programa acaba ficando travado

Saídas (antes e depois da correção)

```
IMPASSE: 2 threads presas
```

```
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

as duas threads ficaram presas porque a thread A estava com o recurso X e esperando o Y, enquanto a thread B estava com o recurso Y e esperando o X. como nenhuma liberava o recurso que a outra precisava, as duas ficaram

esperando e não conseguiram continuar

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

porque quando as duas threads pegam os recursos na mesma ordem, uma espera a outra terminar, evitando que fiquem presas esperando uma pela outra. um exemplo seria um sistema bancário, em que duas transferências precisam acessar as mesmas contas ao mesmo tempo. se não houver uma ordem definida, o sistema pode acabar travando

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

acredito que com 3 trabalhadores vai levar cerca de 200 ms, porque as tarefas serão divididas entre eles. com apenas 1 trabalhador levaria 600 ms, já que ele teria que fazer tudo sozinho. já com 6 trabalhadores levaria cerca de 100 ms, porque cada um poderia realizar uma tarefa ao mesmo tempo

Tempo com 1 trabalhador

1 trabalhador: 600 ms

Tempo com 3 trabalhadores

3 trabalhadores: 200 ms

Tempo com 6 trabalhadores

6 trabalhadores: 100 ms

Saída do programa

com 1 trabalhador, o programa demora cerca de 600 ms porque ele precisa realizar as 6 tarefas sozinho. com 3 trabalhadores, demora aproximadamente 200 ms, já que as tarefas são divididas entre eles. com 6 trabalhadores, leva cerca de 100 ms, pois todos conseguem trabalhar ao mesmo tempo. o programa não precisa de `synchronized` nem de trava porque as tarefas são distribuídas por canais, sem precisar compartilhar uma variável que todos modificam.

5.1 Explique os três tempos. Por que este programa não precisa de `synchronized` nem de trava?

com 1 trabalhador, o programa leva cerca de 600 ms porque ele faz todas as tarefas sozinho. Com 3 trabalhadores, leva 200 ms, pois as tarefas são divididas entre eles. já com 6 trabalhadores, leva 100 ms, porque todos trabalham ao mesmo tempo. não precisa de `synchronized` nem de trava porque as tarefas são distribuídas por canais, sem compartilhar uma variável que todos precisam alterar

Saídas (antes e depois da troca)

```
fatal error: all goroutines are asleep - deadlock!
```

```
1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

o primeiro programa trava porque o canal não tem espaço para armazenar a mensagem e fica esperando alguém receber, mas não existe ninguém para isso. quando colocamos espaço para 1 mensagem, o valor consegue ser armazenado no canal sem precisar esperar. Assim, o programa continua normalmente e imprime o número 1

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

não respondido

Saídas (o erro e o resultado com move)

acho que o programa vai apresentar um erro, porque a thread está tentando alterar uma variável que pertence à função main, sem nenhum tipo de proteção

Saída do programa

```
error[E0373]: closure may outlive the current function,  
but it borrows `contador`
```

```
0
```

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

porque o move faz a thread receber uma cópia do valor da variável contador. assim, a thread soma 1 na sua própria cópia enquanto a variável original da função main continua com o valor 0

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

porque o move faz a thread receber uma cópia do valor da variável contador. assim, a thread soma 1 na sua própria cópia, enquanto a variável original da função main continua com o valor 0.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

achí que a ordem será A, D, C e B porque o JavaScript executa primeiro o código principal e depois as tarefas que ficam esperando nas filas

Saída do programa

```
A  
D  
C  
B  
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

as letras saíram na ordem A, D, C e B. isso acontece porque o JavaScript primeiro executa o código principal, depois as promessas e por último os timers. mesmo com o tempo 0, o B precisa esperar as outras tarefas terminarem para ser executado

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

o timer demorou mais porque o programa ficou ocupado durante 300 ms e não conseguiu executar outras tarefas nesse período. em um servidor Node.js, isso pode ser um problema, porque um cálculo muito pesado pode

bloquear o atendimento dos outros usuários, deixando o sistema lento ou até sem responder por um tempo

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo.

Qual problema desta atividade pode acontecer? Como você o evitaria?

pode acontecer uma condição de corrida, em que as duas pessoas conseguem comprar o mesmo lugar porque o sistema verifica a disponibilidade ao mesmo tempo. para evitar isso, seria necessário usar sincronização, garantindo que apenas uma compra seja realizada por vez

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

o que mais me surpreendeu foi perceber que um programa pode funcionar corretamente em uma execução e apresentar erros em outra mesmo sem mudar nada no código.